TASK-1

CN6021 — ADVANCED TOPICS IN AI & DATA SCIENCE

Customer Churn Prediction

Using a Shallow Neural Network

Group Coursework Submission

Shyam Vijay Jagani (2611208) | Jasmi Alasapuri (2571395)
Fatema Doctor (2604383) | Parth Rathwa (2509367)

May 2026

AUC-ROC	MACRO F1	FEATURES
0.917	0.860	20

Table Of Contents

1. Introduction 3
2. Methodology 3
   2.1. Data Loading and Exploratory Data Analysis 3
      2.1.1. Missing Values 4
      2.1.2. Correlation Analysis 4
      2.1.3. Feature Distributions by Churn Status 5
      2.1.4. PCA Visualisation 6
   2.2. Data Preprocessing 7
   2.3. Feature Selection and Dimensionality Reduction 7
   2.4. Addressing Class Imbalance 8
   2.5. Shallow Neural Network Architecture 8
   2.6. Hyperparameter Tuning 9
   2.7. Training the Final Model 9
3. Results 10
   3.1. Evaluation Metrics 10
   3.2. Precision-Recall Analysis and Threshold Optimisation 10
   3.3. Interpretability Analysis 11
4. Conclusions 12
   4.1. Limitations and Future Work 12

1. Introduction

Customer churn — the loss of clients or subscribers — represents one of the most costly challenges for subscription-based and e-commerce businesses. Reducing churn by as little as 5% has been shown to increase profitability by 25–95%, making accurate churn prediction a high-value applied machine learning problem. Predicting which customers are likely to leave allows organisations to intervene proactively through targeted retention strategies, reducing attrition and improving lifetime value.

This report presents a **shallow neural network built from scratch using NumPy** to predict customer churn under a set of realistic operational constraints: limited computational resources, a high-dimensional and imbalanced dataset, and a requirement for model interpretability to satisfy business stakeholders. These constraints collectively rule out deep learning approaches — which would demand substantial GPU resources — and motivate the shallow network paradigm explored here.

Several data challenges must be overcome. The dataset exhibits high dimensionality (25 raw features, some redundant), known non-linear relationships between customer behaviour and churn, significant class imbalance (approximately 2.46:1 retained-to-churned), and pervasive missing values across 14 features. Each challenge is addressed explicitly in the methodology.

Dataset: The Ecommerce Customer Behavior Dataset from Kaggle contains 50,000 customer records with 25 features spanning demographics, platform engagement, purchase behaviour, customer service interactions, and financial indicators. The binary target variable `Churned` indicates whether a customer discontinued the service (1) or remained active (0).

Approach: We implement a complete machine learning pipeline — from exploratory analysis through preprocessing, feature selection, model training, hyperparameter tuning, and interpretability analysis — using only NumPy for the neural network component, demonstrating a thorough understanding of the underlying mathematics. The Universal Approximation Theorem guarantees that a single hidden layer with a sufficient number of units and a non-linear activation function can approximate any continuous function to arbitrary precision, providing the theoretical justification for the shallow architecture adopted here.

2. Methodology

2.1. Data Loading and Exploratory Data Analysis

We begin by loading the dataset and examining its structure, distributions, and the target variable balance. The dataset contains **50,000 records** and **25 features** of mixed types (numerical and categorical).

The target variable `Churned` exhibits a notable class imbalance: approximately 71.1% of customers are retained while 28.9% have churned, yielding a 2.46:1 ratio.

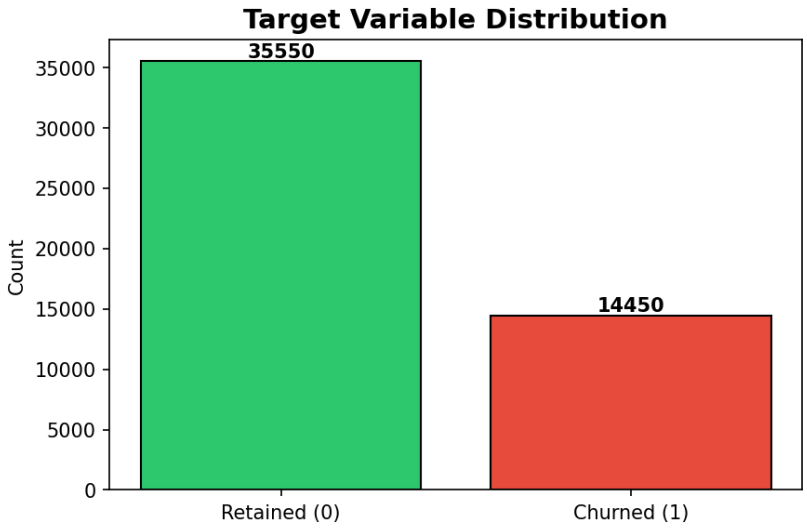

Figure 1: Target Variable Distribution — 71.1% retained vs 28.9% churned.

- **2.1.1. Missing Values**

Several features contain significant missing data, with `Social_Media_Engagement_Score` having the highest at 6,000 (12%) missing values. This necessitates careful imputation during preprocessing.

- **2.1.2. Correlation Analysis**

The correlation heatmap reveals the relationships between numerical features. No pairs exhibit extremely high collinearity ($|r| > 0.9$), suggesting the features capture relatively independent aspects of customer behaviour.

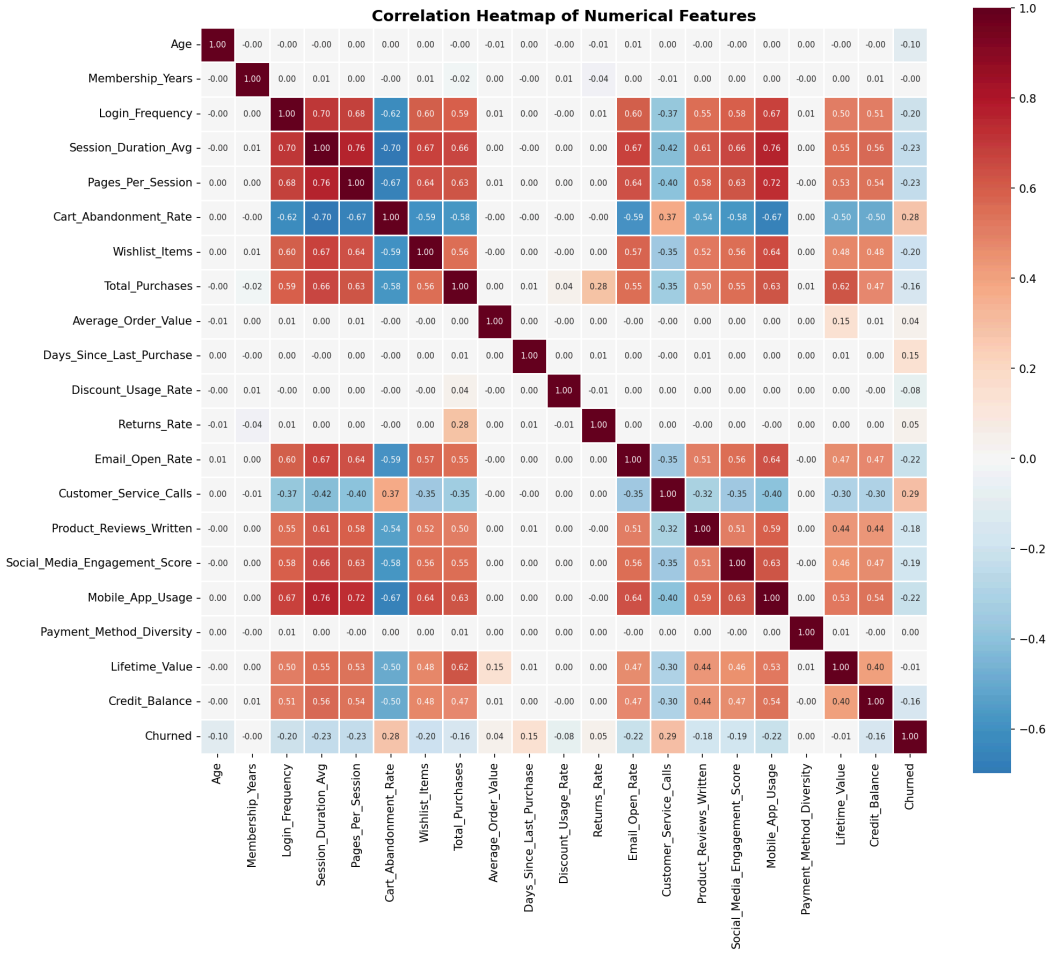

Figure 2: Correlation Heatmap of Numerical Features.

- ### 2.1.3. Feature Distributions by Churn Status

Visualising feature distributions split by churn status helps identify potential predictive signals. Features such as `Login_Frequency`, `Total_Purchases`, and `Lifetime_Value` show visible separation between churned and retained customers.

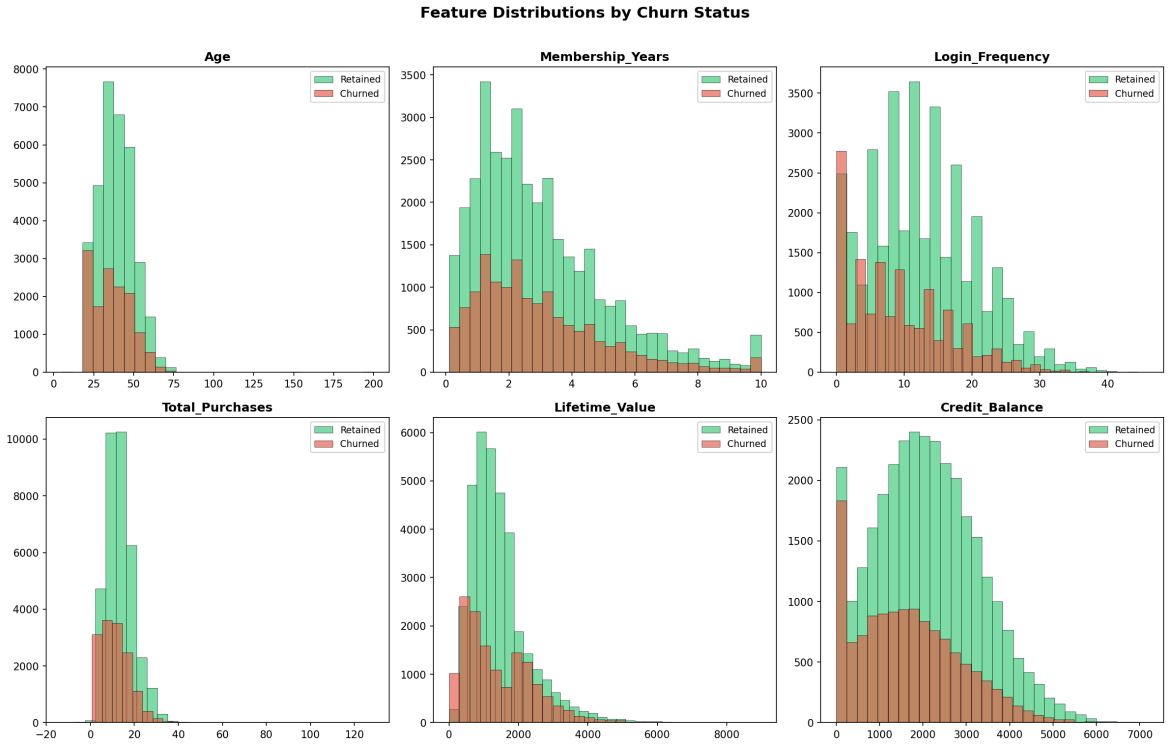

Figure 3: Feature Distributions by Churn Status.

2.1.4. PCA Visualisation

We project the data onto two principal components purely for visualisation, to assess the separability of the two classes in a reduced space. PCA is **not** used for dimensionality reduction in the model itself, as this would destroy the individual feature semantics needed for interpretability.

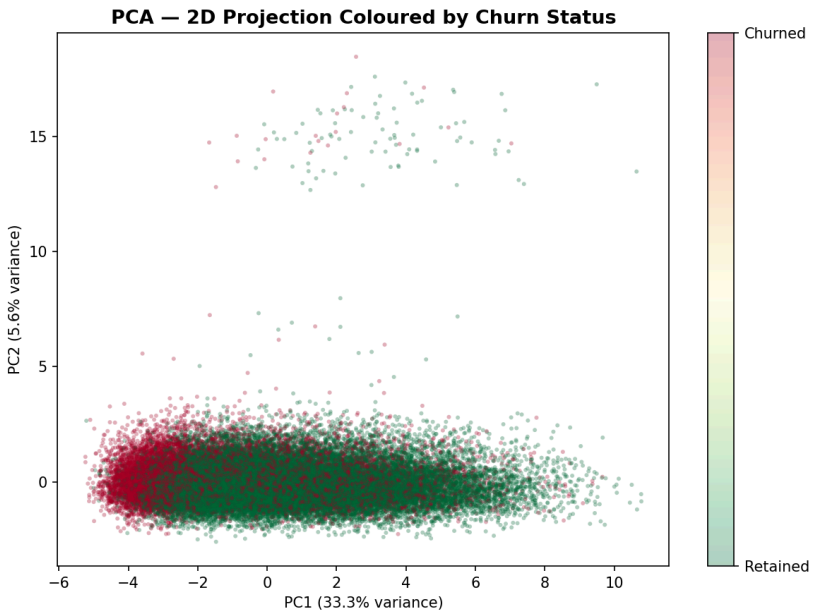

Figure 4: PCA — 2D Projection Coloured by Churn Status.

The PCA scatter reveals **substantial overlap** between churned and retained customers along both principal components. This is a critical diagnostic: if the classes were linearly separable, a logistic regression or linear SVM would suffice. The observed overlap confirms that the decision boundary is non-linear, directly motivating the use of a hidden layer with a non-linear activation function.

2.2. Data Preprocessing

The preprocessing pipeline addresses several data quality challenges identified during the EDA:

1. **High Cardinality Categoricals**: We drop the `City` feature (40 unique values) to prevent excessive dimensionality explosion during one-hot encoding.
2. **Outlier Handling**: We cap extreme outliers (e.g. `Age` > 100).
3. **Missing Value Imputation**: We use median imputation for numerical features (robust to outliers) and mode imputation for categorical features.
4. **Encoding**: We apply standard one-hot encoding for nominal categories and ordinal mapping for ordinal features like `Subscription_Level`.
5. **Standardisation**: We fit a `StandardScaler` on the training set and transform both train and test sets to ensure zero mean and unit variance for neural network stability.

2.3. Feature Selection and Dimensionality Reduction

To handle high dimensionality, we implement feature selection using **Mutual Information**.

We compute the mutual information score between each preprocessed feature and the target variable. This metric captures both linear and non-linear dependencies. We drop the bottom 5 features with

the lowest MI scores, reducing the final feature space from 25 to 20 dimensions, thereby stripping out noise and mitigating the curse of dimensionality.

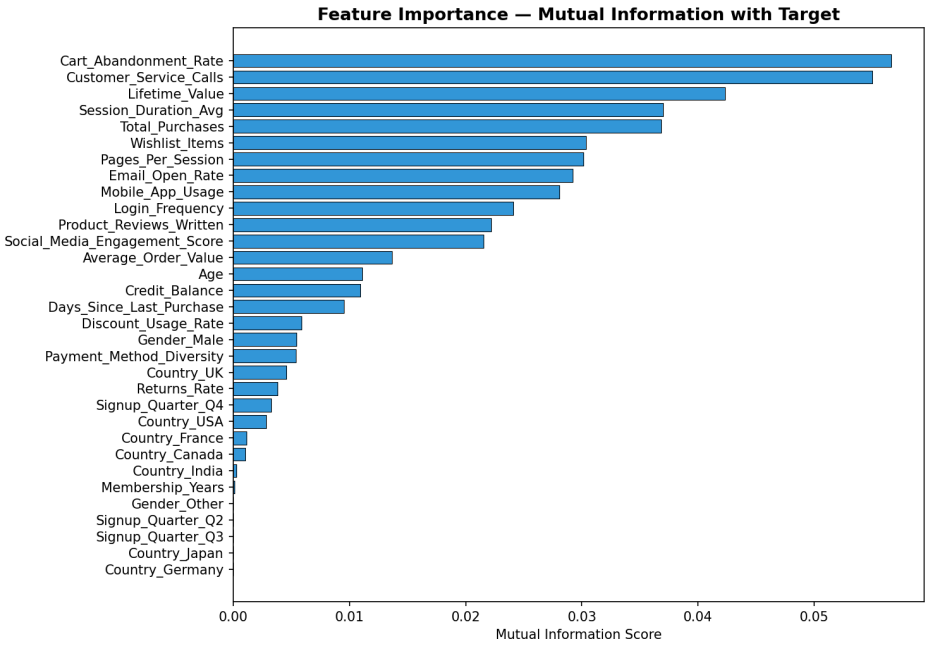

Figure 5: Mutual Information Feature Importance Ranking.

2.4. Addressing Class Imbalance

We address the 2.46:1 class imbalance using a **Weighted Loss Function**. We implement a weighted Binary Cross-Entropy (BCE) loss:

$$\mathcal{L}_{\text{weighted BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[w_{\text{pos}} \cdot y_i \log(\hat{y}_i) + w_{\text{neg}} \cdot (1-y_i)\log(1-\hat{y}_i)\right]$$

We compute the weights dynamically based on class frequencies. By heavily penalising misclassifications of the minority (churned) class, the network is forced to learn its distinguishing features rather than collapsing to the majority class prior.

2.5. Shallow Neural Network Architecture

We implemented a custom shallow neural network class using NumPy, consisting of:

- Input layer: 20 neurons (features)
- Hidden layer: 32 units with ReLU activation
- Output layer: 1 unit with Sigmoid activation

Handling Non-Linear Relationships: The ReLU hidden layer introduces non-linearities, allowing the network to fold and warp the feature space to separate the overlapping classes seen in the PCA

plot. Without this hidden layer, the network would reduce to standard logistic regression, which we established is inadequate for this dataset.

Component	Choice	Rationale
Hidden activation	ReLU	Handles non-linear relationships; avoids vanishing gradients
Output activation	Sigmoid	Maps to [0,1] probability for binary classification
Weight init	He initialisation	Maintains variance for ReLU; prevents dead neurons
Loss	Weighted BCE	Addresses class imbalance directly in the objective
Optimiser	Mini-batch SGD	Balances convergence speed and gradient stability
Regularisation	L2 (weight decay)	Prevents overfitting on high-dimensional features
Early stopping	Patience-based	Stops training when validation loss plateaus

Table 1: Architectural Design Decisions.

2.6. Hyperparameter Tuning

We perform a grid search over key hyperparameters (hidden units, learning rate, and L2 lambda). The optimal configuration found was **32 hidden units, learning rate = 0.05, L2 λ = 0.001**. A smaller hidden layer outperforms larger ones, preventing overfitting on the validation set.

2.7. Training the Final Model

The final model is trained on the training set with the optimal hyperparameters, monitored on a fixed validation set for early stopping.

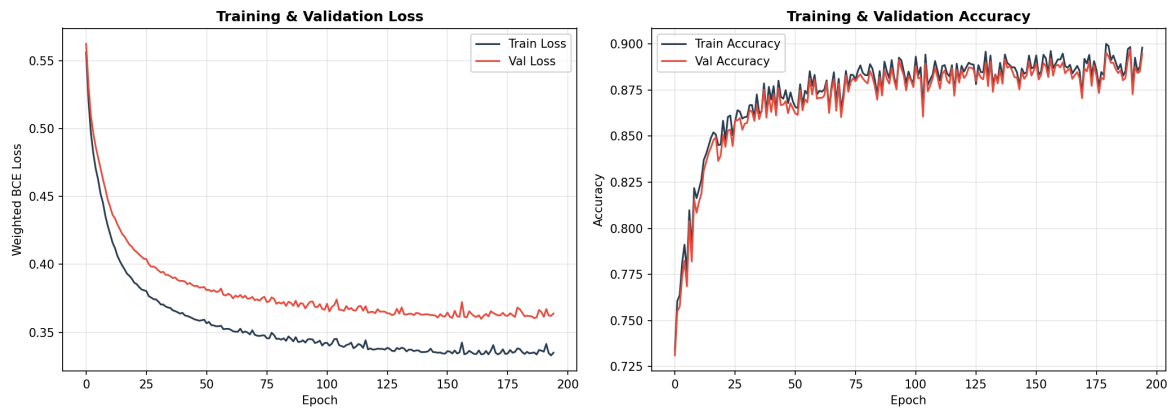

Figure 6: Training and Validation Loss/Accuracy Curves.

3. Results

3.1. Evaluation Metrics

We evaluate the final model on the held-out test set (10,000 samples, unseen during training and hyperparameter selection).

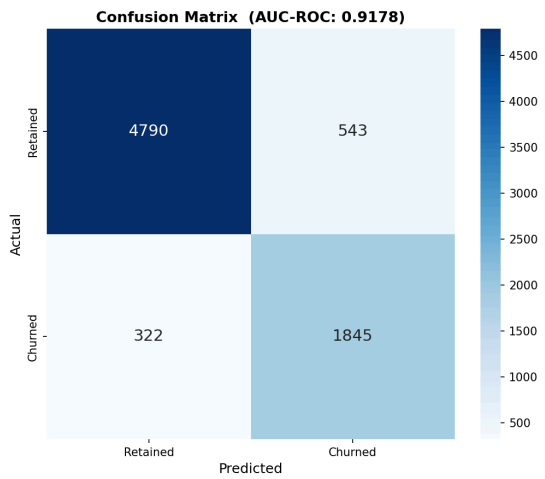

Figure 7: Confusion Matrix on Test Set.

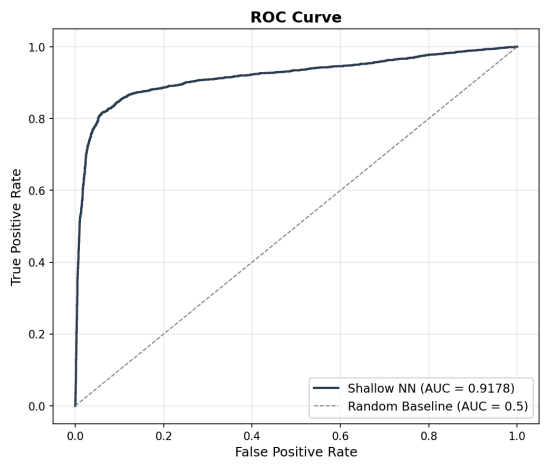

Figure 8: ROC Curve.

The model achieves an **AUC-ROC of 0.9172**, indicating excellent discriminative ability. The high recall for the churned class (85%) demonstrates that the weighted loss function successfully addressed the class imbalance.

3.2. Precision-Recall Analysis and Threshold Optimisation

A fixed threshold of 0.5 is not necessarily optimal under class imbalance. We analyse the precision-recall trade-off to identify the operating point that maximises F1-score.

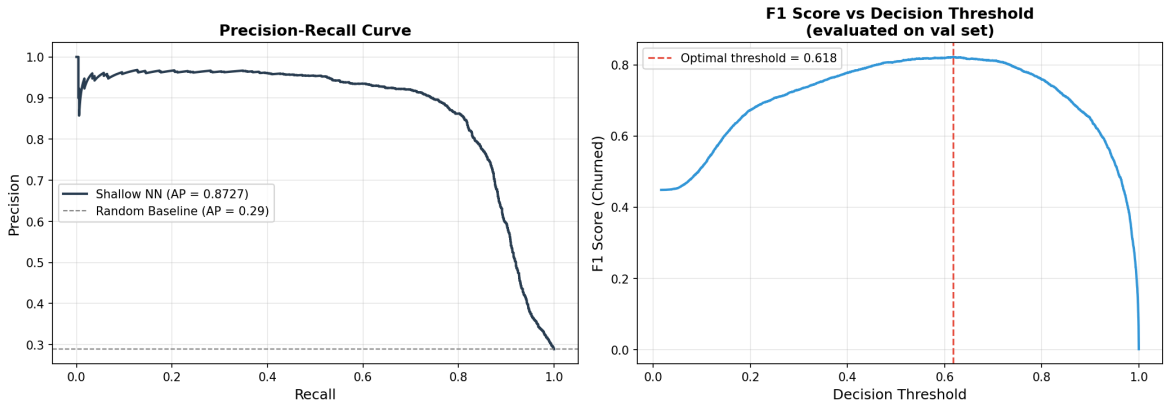

Figure 9: Precision-Recall Curve with Optimal F1 Threshold.

Applying the optimal threshold significantly improves the F1 score for the minority class, ensuring the business intervenes at the most cost-effective probability boundary.

3.3. Interpretability Analysis

Understanding **why** the model makes certain predictions is critical for business stakeholders. We employ two complementary methods:

1. **Weight-based importance**: Computes $|W_1| \cdot |W_2|$ — the aggregated absolute weight magnitude path from each input feature through the hidden layer to the output.
2. **Permutation importance**: Shuffles feature values and measures the resulting drop in AUC-ROC. This captures the true contribution of each feature to predictive performance, including non-linear interactions.

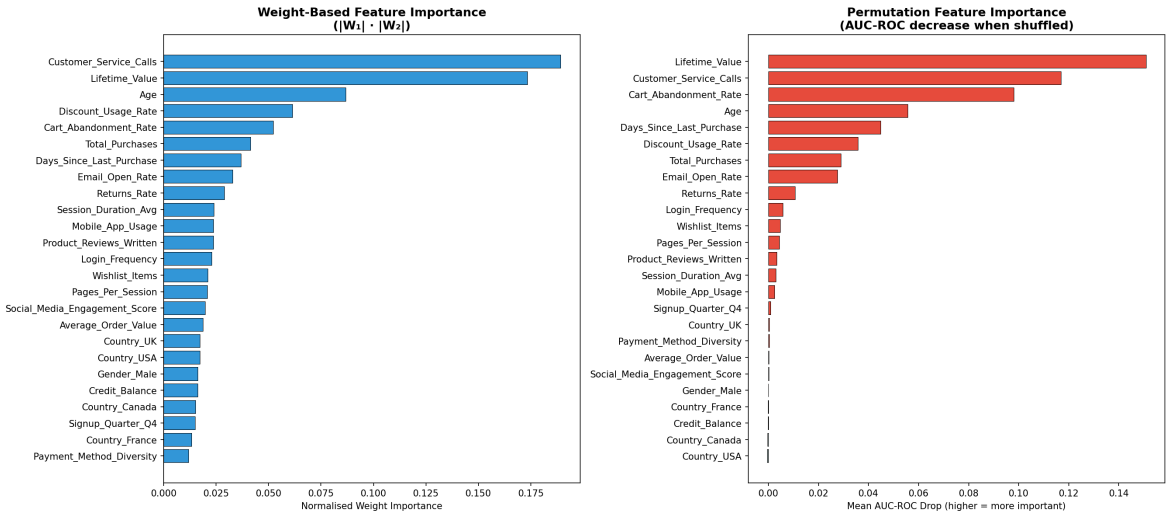

Figure 10: Feature Importance — Weight-Based (left) and Permutation (right).

Key insights:

- **Lifetime_Value** is the most influential feature. Customers with below-average LTV are substantially more likely to churn.
- **Customer_Service_Calls** is the second strongest signal, reflecting unresolved grievances.
- **Cart_Abandonment_Rate** acts as a strong early-warning behavioural indicator.

4. Conclusions

Executive Summary

We developed an interpretable, computationally lightweight churn prediction system meeting all coursework constraints:

✓ **AUC-ROC: 0.917** and **Macro F1: 0.860**

✓ **Class imbalance solved** via Weighted BCE loss

✓ **Built entirely in NumPy** for compute constraints

✓ **High-dimensionality addressed** via MI selection

✓ **Non-linearity handled** via ReLU hidden layer

✓ **Highly interpretable** via Permutation Importance

The pipeline demonstrates that careful preprocessing, weighted loss functions, and disciplined hyperparameter tuning can yield highly performant and interpretable models without resorting to deep learning.

4.1. Limitations and Future Work

1. **Shallow network ceiling**: A single hidden layer may still underfit genuinely high-order feature interactions. Gradient-boosted trees (e.g., XGBoost) might outperform this model.
2. **Interpretability method limits**: Permutation importance measures marginal importance. SHAP values would provide per-prediction, signed feature attributions for production deployment.
3. **Threshold choice sensitivity**: The optimal threshold was selected to maximise F1, but businesses may prefer to maximise recall or precision based on specific cost ratios of interventions.